



CS 2201

Computer Algorithms

Solving Recurrences Continued

The Master Theorem

Review: Solving Recurrences

- Substitution method
- Iteration method
- Master method

Review: Solving Recurrences

- The substitution method
 - The “making a good guess method”
 - Guess the form of the answer, then use induction to find the constants and show that solution works
 - *Run an example*: merge sort
 - $T(n) = 2T(n/2) + cn$
 - We guess that the answer is $O(n \lg n)$
 - Prove it by induction
 - Can similarly show $T(n) = \Omega(n \lg n)$, thus $\Theta(n \lg n)$

Review: Solving Recurrences

- The “iteration method”
 - Expand the recurrence
 - Work some algebra to express as a summation
 - Evaluate the summation
- We showed several examples, were in the middle of:

$$T(n) = \begin{cases} c & n = 1 \\ aT\left(\frac{n}{b}\right) + cn & n > 1 \end{cases}$$

$$T(n) = \begin{cases} c & n = 1 \\ aT\left(\frac{n}{b}\right) + cn & n > 1 \end{cases}$$

- $T(n) =$
 $aT(n/b) + cn$
 $\dots$
 $a^k T(n/b^k) + cn(a^{k-1}/b^{k-1} + a^{k-2}/b^{k-2} + \dots + a^2/b^2 + a/b + 1)$

$$T(n) = \begin{cases} c & n = 1 \\ aT\left(\frac{n}{b}\right) + cn & n > 1 \end{cases}$$

- So we have
 - $T(n) = a^k T(n/b^k) + cn(a^{k-1}/b^{k-1} + \dots + a^2/b^2 + a/b + 1)$
- For $k = \log_b n$
 - $n = b^k$
 - $T(n) = cn(a^k/b^k + \dots + a^2/b^2 + a/b + 1)$

$$T(n) = \begin{cases} c & n = 1 \\ aT\left(\frac{n}{b}\right) + cn & n > 1 \end{cases}$$

- So with $k = \log_b n$
 - $T(n) = cn(a^k/b^k + \dots + a^2/b^2 + a/b + 1)$
- What if $a = b$?
 - $T(n) = cn(k + 1)$
 $= \Theta(n \log n)$

$$T(n) = \begin{cases} c & n = 1 \\ aT\left(\frac{n}{b}\right) + cn & n > 1 \end{cases}$$

- So with $k = \log_b n$
 - $T(n) = cn(a^k/b^k + \dots + a^2/b^2 + a/b + 1)$
- What if $a < b$?

$$T(n) = \begin{cases} c & n = 1 \\ aT\left(\frac{n}{b}\right) + cn & n > 1 \end{cases}$$

- So with $k = \log_b n$

- $T(n) = cn(a^k/b^k + \dots + a^2/b^2 + a/b + 1)$

- What if $a < b$?

- So:

$$\frac{a^k}{b^k} + \frac{a^{k-1}}{b^{k-1}} + \dots + \frac{a}{b} + 1 = \frac{(a/b)^{k+1} - 1}{(a/b) - 1} = \frac{1 - (a/b)^{k+1}}{1 - (a/b)} < \frac{1}{1 - a/b}$$

- $T(n) = cn \cdot \Theta(1) = \Theta(n)$

$$T(n) = \begin{cases} c & n = 1 \\ aT\left(\frac{n}{b}\right) + cn & n > 1 \end{cases}$$

- So with $k = \log_b n$
 - $T(n) = cn(a^k/b^k + \dots + a^2/b^2 + a/b + 1)$
- What if $a > b$?

$$T(n) = \begin{cases} c & n = 1 \\ aT\left(\frac{n}{b}\right) + cn & n > 1 \end{cases}$$

- So with $k = \log_b n$

- $T(n) = cn(a^k/b^k + \dots + a^2/b^2 + a/b + 1)$

- What if $a > b$?

$$\frac{a^k}{b^k} + \frac{a^{k-1}}{b^{k-1}} + \dots + \frac{a}{b} + 1 = \frac{(a/b)^{k+1} - 1}{(a/b) - 1} = \Theta((a/b)^k)$$

- $T(n) = cn \cdot \Theta(a^k / b^k)$
 $= cn \cdot \Theta(a^{\log n} / b^{\log n}) = cn \cdot \Theta(a^{\log n} / n)$
 $= \Theta(n^{\log a})$

$$T(n) = \begin{cases} c & n = 1 \\ aT\left(\frac{n}{b}\right) + cn & n > 1 \end{cases}$$

• So...

$$T(n) = \begin{cases} \Theta(n) & a < b \\ \Theta(n \log_b n) & a = b \\ \Theta(n^{\log_b a}) & a > b \end{cases}$$

The Master Theorem

- Given: a *divide and conquer* algorithm
 - An algorithm that divides the problem of size n into a subproblems, each of size n/b
 - Let the cost of each stage (i.e., the work to divide the problem + combine solved subproblems) be described by the function $f(n)$
- Then, the Master Theorem gives us a cookbook for the algorithm's running time:

The Master Theorem

- if $T(n) = aT(n/b) + f(n)$ then

$$T(n) = \left\{ \begin{array}{ll} \Theta(n^{\log_b a}) & f(n) = O(n^{\log_b a - \varepsilon}) \\ \Theta(n^{\log_b a} \log n) & f(n) = \Theta(n^{\log_b a}) \\ \Theta(f(n)) & f(n) = \Omega(n^{\log_b a + \varepsilon}) \text{ AND} \\ & af(n/b) < cf(n) \text{ for large } n \end{array} \right\} \begin{array}{l} \varepsilon > 0 \\ c < 1 \end{array}$$

Using The Master Method

- $T(n) = 9T(n/3) + n$
 - $a=9, b=3, f(n) = n$
 - $n^{\log_b a} = n^{\log_3 9} = \Theta(n^2)$
 - Since $f(n) = O(n^{\log_3 9 - \epsilon})$, where $\epsilon=1$, case 1 applies:
$$T(n) = \Theta(n^{\log_b a}) \text{ when } f(n) = O(n^{\log_b a - \epsilon})$$
 - Thus the solution is $T(n) = \Theta(n^2)$